PSOC™ Control C3 (P/M) 6 MCU provisioning specification

About this document

Scope and purpose

This application note provides essential information for enabling the provisioning process of Infineon's PSOC™ Control C3 (P/M) 6 devices. It details the RAM application, Edge Protect Tools, and policy information.

Intended audience

This document is intended for users with advanced knowledge of the various components involved in the PSOC™ Control C3 (P/M) 6 provisioning process.

This document presumes that you are already familiar with AN243049 – Getting started with PSOC™ Control C3 (P/M) 6 MCU on ModusToolbox™ software and AN243370 – Getting started with PSOC™ Control C3 (P/M) 6 security.

Table of contents

About this document . . . 1

Table of contents . . . 2

1 Introduction . . . 4

1.1 Provisioning overview . . . 4

1.2 Re-provisioning . . . 6

2 Certificates . . . 7

2.1 OEM certificate . . . 7

2.2 OEM CSR . . . 8

3 Policy . . . 11

3.1 Debug policy . . . 12

3.2 Boot policy . . . 14

3.3 DFU policy . . . 16

3.4 Key-revocation policy . . . 19

3.5 Finalizing OEM assets . . . 19

3.6 Lifecycle stage (LCS) transitioning . . . 20

4 Edge Protect Tools . . . 21

4.1 Overview . . . 21

4.2 Device communication . . . 22

4.3 Edge Protect Tools package for PSOC™ Control C3 (P/M) 6 MCU . . . 22

4.3.1 Provisioning the device . . . 24

4.3.2 Packaged assets . . . 25

5 Downloadable Module (DLM) package . . . 26

5.1 RAM application (RAM_APP) . . . 26

5.2 Downloadable Module (DLM) package . . . 26

5.3 Creating a DLM package . . . 29

5.4 Download and launch of the DLM package . . . 29

5.4.1 Launch flow: BootROM DFU . . . 30

A Appendix . . . 31

A.1 RAM application (DLM) status codes . . . 31

A.2 Edge Protect Tools supported commands . . . 33

A.3 CBOR and COSE format . . . 34

References . . . 35

Glossary . . . 36

Revision history . . . 39

Trademarks . . . 40

Table of contents

Disclaimer 41

1 Introduction

Provisioning is the process of injecting authorized keys and policy configuration into a device. Original equipment manufacturers (OEMs) use the provisioning process to load OEM root of trust (RoT) keys and OEM policies into the PSOC™ Control C3 (P/M) 6 device. The provisioning of OEM keys, certificates, and policies transfers device ownership from Infineon to the OEM.

PSOC™ Control C3 (P/M) 6 devices can be provisioned using Infineon's Edge Protect Tools package, which creates and downloads a provisioning packet into the device. This provisioning packet includes a RAM application that can perform the provisioning, along with input parameters for the RAM application, such as the policy file and keys.

This document provides detailed information about the various types of certificates, policies, RAM application, and downloadable module (DLM) packages, all of which are included in or generated by Edge Protect Tools.

This document is not a programming specification. Refer to AN238998 - PSOC™ Control C3 MCU programming specification.

1.1 Provisioning overview

The provisioning process enables the OEM to configure several boots and debug related policy configurations. As a primary step, the OEM is required to ‘own’ the device before making any changes to the Infineon default policies on the device. To take ownership of the device, the OEM needs to generate its own keys and certificate, and provision the device with these, as shown in Step 1 of Figure 1. This step can be performed whenever the OEM needs to change the root of trust (RoT) keys on the device, even while the device is in the development lifecycle stage (LCS). After the transfer of ownership is completed, the OEM must then use the OEM_ROT_KEY to perform further provisioning of the device for the purpose of policy modification as shown in Step 2 of Figure 1. There are no restrictions on the number of times the device can be provisioned in the development lifecycle stage. Therefore, provisioning can be utilized to carry out the following operations on the device:

- Replace keys and certificates
- Replace the OEM policy file

The device can then be provisioned 'one' time to transition into the production lifecycle by using the production certificate. After the device is moved into the production lifecycle stage, it cannot be returned to the development LCS and the OEM_ROT_KEY can no longer be changed through provisioning. In the production lifecycle stage (LCS), you only have a single opportunity to perform key revocation, which allows you to replace the existing OEM ROT keys with new ones. This operation is irreversible and can only be performed once.

1 Introduction

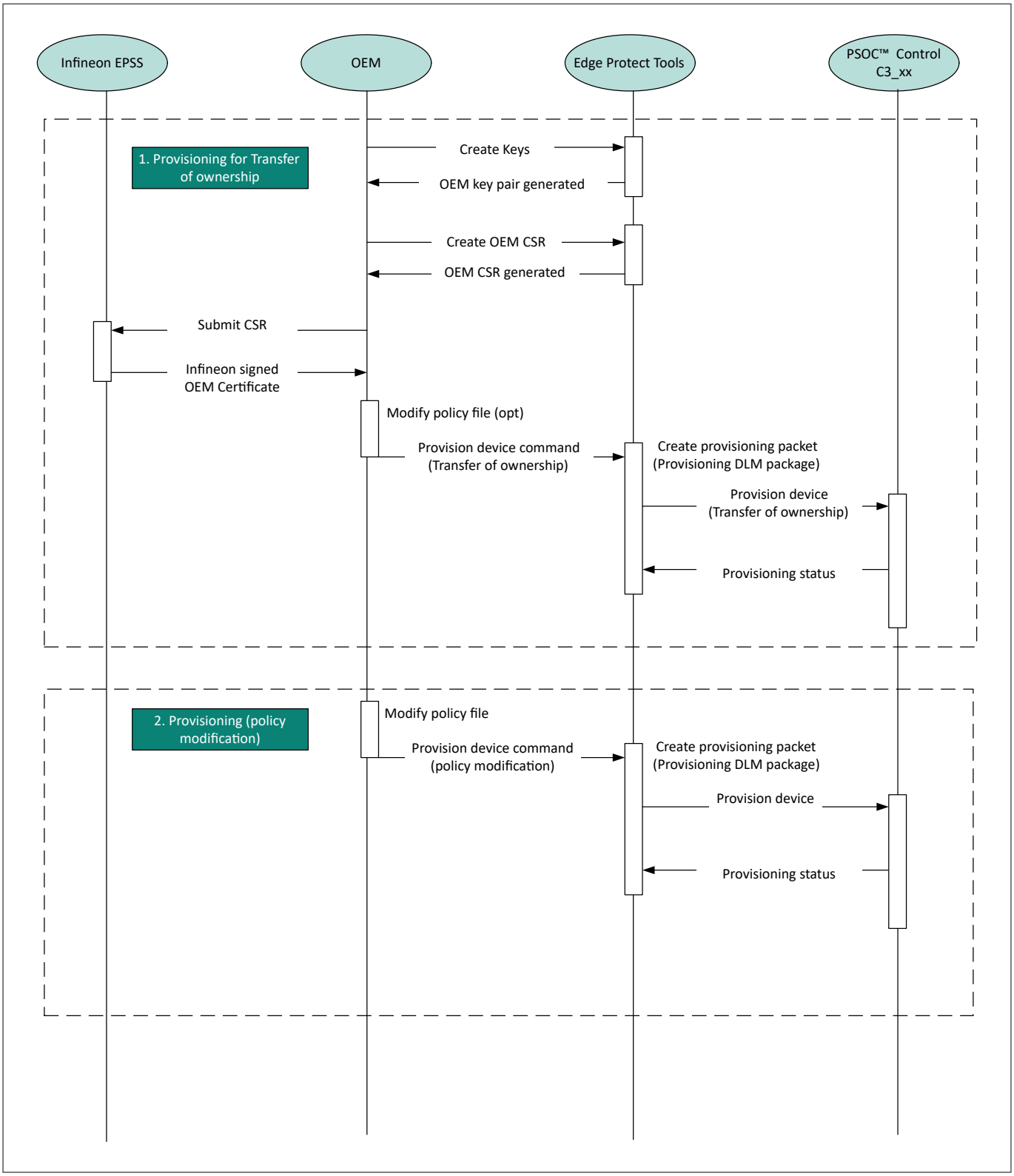


Figure 1 Provisioning overview

***Note**: To program an eFuse, VDDIO0 must be at 2.5 V ±10%. See the datasheet for more details.*

1.2 Re-provisioning

In the production LCS, re-provisioning can be performed any number of times as long as the `prov_oem_complete` flag is set to "false" in the OEM provisioning policy (`/policy/policy_oem_provisioning.json`). Once the `prov_oem_complete` is set to "true", the contents in SFlash are locked and no further provisioning is possible (Refer to section Finalizing OEM assets). Note that this behavior applies to both the Development and Production lifecycle stages.

2 Certificates

A digital certificate or a public-key certificate can be used by any entity to establish the ownership of a public key. Certificates contain a public key, information about the public key, identity of its owner, and the signature of the certificate authority (CA) that verifies the content of the certificate. A certificate signing request (CSR) is a request sent from the entity that owns the public key to the certificate authority to verify the information provided in the CSR and generate the certificate.

There are different kinds of certificates and CSR that are used in the context of PSOC™ Control C3x6 devices.

Certificate	Purpose
OEM certificate	This contains the OEM_ROT_PUBLIC_KEY, which is signed by the Infineon CA and provisioned by the OEM to transfer the ownership of the device. See OEM certificate
OEM CSR	This is a self-signed certificate signing request sent from the OEM to the Infineon CA to generate the OEM certificate. See OEM CSR

For PSOC™ Control C3x6 devices, the certificates use the CBOR/COSE format. The device ownership is transferred to the OEM when the device is provisioned with the OEM certificate. This certificate is signed by Infineon through Edge Protect Signing Service (EPSS) and is verified by the device after provisioning. This process ensures that only Infineon approved OEM certificate can be provisioned on the device. OEM certificate contains the OEM root of trust public Key. The OEM root of trust private key must be protected by the OEM and used during provisioning of the device to sign the OEM policy, and also to sign the user application images before programming, in case secure boot is enabled.

2.1 OEM certificate

The OEM certificate contains the OEM_ROT_PUBLIC_KEY and enables the OEM to take ownership of the device. This certificate must be signed by the IFX_ROT_KEY, and generated by Infineon's Edge Protect Signing Service (EPSS) based on the CSR uploaded by the OEM. For more detailed instructions on generating the OEM CSR and OEM certificate, see AN243370 – Getting started with PSOC™ Control C3 (P/M) 6 security.

The OEM certificate can be of two types:

- **Development certificate:** The development OEM certificate contains the OEM_DEV_PUBLIC_KEY that is used by the OEM during the development LCS of the device. In the development lifecycle, the device can be provisioned multiple times to change the policy/keys, and can also be restored to Infineon default policy settings. When the development certificate is used, the keys are stored in a mutable memory and the lifecycle of the device remains unchanged. Development OEM keys do not need to be secured and can be replaced at any time by the OEM
- **Production certificate:** The production OEM certificate is used to transition the device from the development to production LCS. This certificate must contain the production OEM_ROT_PUBLIC_KEY that will be stored in the immutable memory region. Two sets of the OEM_ROT_KEYs are supported to enable key revocation in the event of a security breach. The production OEM ROT keys, used for production devices, must be securely stored and used by the OEM. These keys cannot be replaced by provisioning

Table 1 OEM certificate fields

Item	Field	Example value	Data type	Description
1	VERSION	1	String	Certificate version
2	TEMPLATE_TYPE	OEM_CERT	String	Certificate template
3	CERTIFICATE_NAME	OEM_CSR	String	Certificate name

(table continues...)

Table 1 (continued) OEM certificate fields

Item	Field	Example value	Data type	Description
4	TYPE	DEVELOPMENT	String	Certificate type. Applicable values: DEVELOPMENT, PRODUCTION
5	ISSUER	Dummy Corp.	String	Certificate issuer
6	ID	ABCDEF	HEX	Certificate (unique) ID
7	DATE	2023-09-19 15:24:47	String	The date of creation of the certificate
8	ALGORITHM	ES521	String	Key algorithm. Applicable values: ES256, ES384, ES521
9	SIGNER_ID	Dummy_OEM	String	Certificate signer ID
10	PUBLIC_KEY_0	049A0BFAC2727E7...	HEX	OEM_ROT public key 0
11	PUBLIC_KEY_1	049A0BFAC2727E7...	HEX	OEM_ROT public key 1
12	OEM	Dummy_OEM	String	OEM name
12	PROJECT	Dummy_Project	String	Project name
13	PROJECT_NUMBER	123	String	Project number

The OEM certificate generated by Infineon's EPSS will be in the COSE format. A JSON representation of the same is shown in the following code block:

OEM certificate in JSON format

```
{
"VERSION": "1",
"TEMPLATE_TYPE": "OEM_CERT",
"CERTIFICATE_NAME": "OEM_CSR",
"TYPE": "DEVELOPMENT",
"ISSUER": "Dummy Corp.",
"ID":h'ABCDEF',
"DATE":"2023-09-19 15:24:47",
"ALGORITHM": "ES521",
"SIGNER_ID": "555",
"PUBLIC_KEY_0":
h'04C55A85F02FDF5D2256397C7C154DAF2AECDBB56403F4ED279D47AACE1D9270063BFB09AB8CFFE8DC2172DC354D37
7495ECFE2FE0C8A55E3B8F521F79FF874CBB',
"PUBLIC_KEY_1":
h'04504D54E877726B856CF59940CCE458DA85F805FE1AC4CDEF692B598B3CE11E772C841F2F6C09998090E5D85C468B
BE85431FE4EFBA6AED8D9777E2E46A92C84D',
```

2.2 OEM CSR

The OEM certificate signing request (CSR) must be generated by the OEM with the OEM_ROT_PUBLIC_KEY and other relevant information. The CSR is signed by the OEM_ROT_PRIVATE_KEY. The OEM CSR can be generated

using the Edge Protect Tools. The OEM CSR must be submitted to Infineon's Edge Protect Signing Service (EPSS) to generate the OEM certificate.

Table 2 OEM CSR fields

Item	Field	Example value	Data type	Description
1	VERSION	1	String	Certificate version
2	TEMPLATE_TYPE	OEM_CSR	String	Certificate type
3	CERTIFICATE_NAME	–	–	Not applicable
4	OEM	Dummy_OEM	String	OEM name
5	PROJECT	Dummy_Project	String	Project name
6	PROJECT_NUMBER	123	String	Project number
7	ISSUER	Dummy Corp.	String	Certificate issuer
8	SIGNER_ID	555	String	Signer ID
9	ALGORITHM	ES521	String	Key algorithm. Applicable values: ES256, ES384, ES521
10	PUBLIC_KEY_0	"keys/ oem_public_key_0.pem"	String	Path to the ECDSA P-256 public key or a 65-byte hex string
11	PUBLIC_KEY_1	"keys/ oem_public_key_1.pem"	String	Path to the ECDSA P-256 public key or a 65-byte hex string
12	TYPE	DEVELOPMENT	String	Certificate type. Applicable values: DEVELOPMENT, PRODUCTION
13	DATE	2023-09-19 15:24:47	String	The date of creation of the certificate in the format: YYYY-MM-DD hh:mm:ss. If not provided, the system time will be used
14	ID	ABCDEF	HEX	Certificate (unique) ID

2 Certificates

The OEM CSR generated by the Edge Protect Tools will be in the CBOR Object Signing and Encryption (COSE) format. A JSON representation of the same is shown in the following code block:

```
{
"VERSION": "1",
"TEMPLATE_TYPE": "OEM_CSR",
"CERTIFICATE_NAME": "",
"OEM": "Dummy_OEM",
"PROJECT": "Dummy_Project",
"PROJECT_NUMBER": "123",
"ISSUER": "Dummy Corp.",
"SIGNER_ID": "555",
"ALGORITHM": "ES521",
"PUBLIC_KEY_0": "keys/oem_public_key_0.pem",
"PUBLIC_KEY_1": "keys/oem_public_key_1.pem",
"TYPE": "DEVELOPMENT",
"DATE": "2023-09-19 15:24:47",
"ID": h'ABCDEF'
}
```

3 Policy

The boot configuration of PSOC™ Control C3 (P/M) 6 devices is dictated by a policy file. All policy configurations are applied during device boot up by the Infineon BootROM (ROM_BOOT + FLASH_BOOT). The policy file is a JSON configuration file that defines the settings/configurations i.e., input parameters for device provisioning. Refer to AN243370: Getting started with PSOC™ Control C3 (P/M) 6 security - Policy file for list of input parameters that can be provisioned to device.

In the Infineon factory, the device is provisioned with a set of default policy configurations. This default policy is packaged as a part of the Edge Protect Tools package and contains different sections that can be broadly classified into the three OEM policy sections:

- Debug policy
- Boot policy
- DFU policy

The policy file also includes a Keys section where the user OEM_ROT_KEY and device parameters are defined. The following figure shows what different input parameters are part of the policy file.

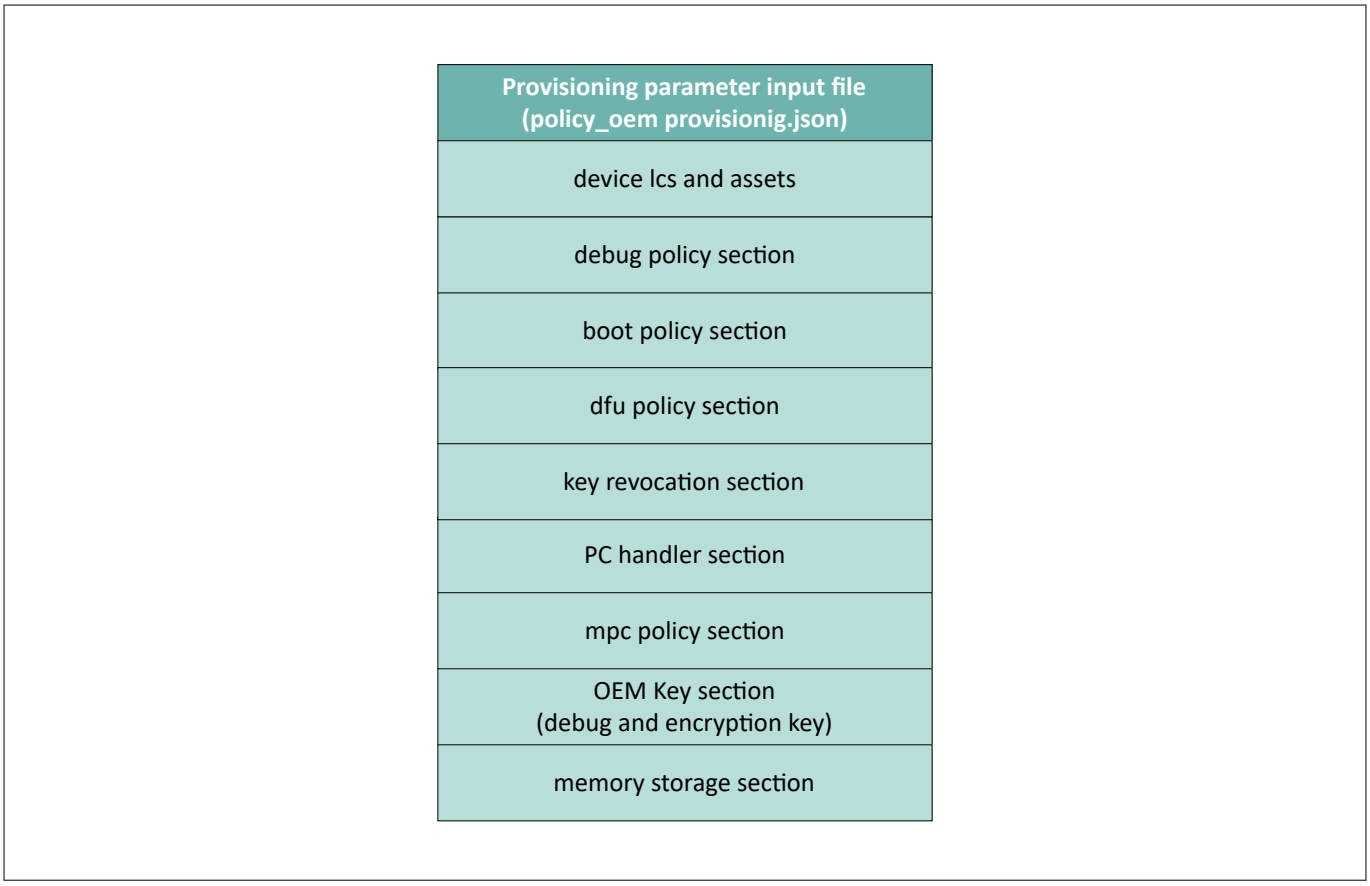


Figure 2 OEM policy file layout

Once the Edge Protect Tools package is initialized on the host PC, see the JSON policy in the `<application-directory>/policy/policy_oem_provisioning.json` file. This file can be edited using any text editor to customize the policy.

The policy file is converted into a CBOR packet that contains all provisioning data by Edge Protect Tools. This packet is then signed accordingly to the COSE format and the final packet is sent to the device during provisioning by Edge Protect Tools. The RAM application on the device then receives the packet, verifies the packet's signature, and processes the CBOR payload to get data for the provisioning. Typically, the RAM application processes provisioning data to convert it to the asset format stored on the device.

3.1 Debug policy

The debug policy configuration is used by the OEM to control access to debug ports through two mechanisms:

- Direct configuration
  - Enable/disable debug ports
  - Infineon's default policy enables all debug configurations
- Authenticated debug
  - Debug ports can be temporarily enabled using a debug token
  - Token must be cryptographically signed with authorized OEM RoT keys
  - Provides secure, controlled debug access when ports are disabled

***Note**: While Infineon's default policy enables all debug configurations, OEMs can implement authenticated debug for enhanced security by requiring valid debug tokens to access disabled ports.*

The authenticated debug works only when the debug field is set to “Permanently Disable” in the policy file. When set to “Disable”, the debug ports are permanently disabled and cannot be enabled even with debug token.

The default configurations in a policy file are shown in the following code block:

3 Policy

Debug default policy configuration


```
"debug": {
      "cpu": {
        "ap_cm33": {
          "description": "Configures the CM33 debug access port availability in the SECURE and
NORMAL_PROVISIONED LCS",
          "applicable_conf": "Enable, Disable, Permanently Disable",
          "value": "Enable"
        },
        "listen_window": {
          "description": "Configures the listen window time to acquire the CM33 debug access
port in the SECURE LCS",
          "applicable_conf": "100 ms, 20 ms, 2 ms, 0 ms",
          "value": "100 ms"
        },
        "cm33_dbg": {
          "description": "Configures the invasive debug of CM33 core",
          "applicable_conf": "Enable, Disable, Permanently Disable",
          "value": "Enable"
        },
        "cm33_nid": {
          "description": "Configures the non-invasive debug of CM33 core",
          "applicable_conf": "Enable, Disable, Permanently Disable",
          "value": "Enable"
        },
        "cm33_spid": {
          "description": "Configures the secure privileged invasive debug of CM33 core",
          "applicable_conf": "Enable, Disable, Permanently Disable",
          "value": "Enable"
        },
        "cm33_spnid": {
          "description": "Configures the secure privileged non-invasive debug of CM33 core",
          "applicable_conf": "Enable, Disable, Permanently Disable",
          "value": "Enable"
        },
        "cm33_secure": {
          "description": "Enables the CM33 secure AP interface",
          "applicable_conf": "Enable, Disable, Permanently Disable",
          "value": "Enable"
        }
      },
      "system": {
        "ap": {
          "description": "Configures the System debug access port availability in the SECURE
and NORMAL_PROVISIONED LCS",
          "applicable_conf": "Enable, Disable, Permanently Disable",
          "value": "Enable"
        },
        "secure": {
          "description": "Enables the system secure AP interface",
          "applicable_conf": "Enable, Disable, Permanently Disable",
          "value": "Enable"
```

```
        },
        "mmio_restrictions": {
          "description": "MMIO restrictions for System AP",
          "applicable_conf": "No restrictions, SRSS_GENERAL2 only, No access",
          "value": "No restrictions"
        },
        "debug_pins": {
          "description": "Configures debug pins and interface",
          "comment_1": "0000 - 2 pins (TMS, TCLK), JTAG is disabled",
          "comment_2": "0011 - 2 pins (TMS, TCLK), JTAG is enabled",
          "comment_3": "0101 - 4 pins (TMS, TCLK, TDO, TDI)",
          "comment_4": "0110 - 5 pins (TMS, TCLK, TDO, TDI, TRSTN)",
          "comment_5": "1001 - SWJ pins disabled, JTAG is enabled",
          "comment_6": "1010 - SWJ pins disabled, JTAG is disabled",
          "applicable_conf": "0000, 0011, 0101, 0110, 1001, 1010",
          "value": "0000"
        }
      }
    },
```

3.2 Boot policy

The boot policy in the OEM provisioning policy defines how the device authenticates and launches its application(s) during startup. It specifies the boot configuration/layout, the image authentication behavior, the signature algorithm used for verification, the application memory slots, and the CM33 boot clock frequency. Together, these settings determine the device's secure boot behavior and can be tailored by the user to match the intended application configuration. The default configurations in a policy file are shown in the following code block:

3 Policy

Boot policy configuration

```
"boot": {
      "boot_cfg_id": {
        "description": "A behavior for BOOT_APP_LAYOUT",
        "applicable_conf": "SIMPLE_APP, SECURE_APP, DUAL_BANK_SIMPLE_APP, DUAL_BANK_SECURE_APP,
PROT_FW",
        "value": "SIMPLE_APP"
      },
      "boot_bank_ctr_offset": {
        "description": "An offset from the start of each flash bank where a 32-bit counter for
flash dual bank switching is placed",
        "value": "0xC39D5455"
      },
      "boot_hash_locking": {
        "description": "Enable hash locking, fast authentication (compare a hash) instead of
ECDSA",
        "value": false
      },
      "boot_auth_images": {
        "description": "Configures the images authentication and moving from a SSS after DFU
update",
        "applicable_conf": "NONE, ONE, ALL, ALL_MOVE",
        "comment_1": "NONE - Disable all authentications",
        "comment_2": "ONE - Authenticate the first image, ignore the other images",
        "comment_3": "ALL - Authenticate all images",
        "comment_4": "ALL_MOVE - Authenticate all images, move from SSS on a successful DFU
update",
        "value": "ALL_MOVE"
      },
      "boot_auth_signature": {
        "description": "Signature type for Boot Authentication",
        "applicable_conf": "ES256, ES384, ES512",
        "value": "ES512"
      },
      "boot_app_layout": {
        "description": "The memory layout for the applications defined by BOOT_CFG_ID.
0x32000000 - 0x33FFFFFF for secure addresses; 0x22000000 - 0x23FFFFFF for non-secure addresses",
        "value": [
          {
            "address": "0x32000000",
            "size": "0x40000"
          },
          {
            "address": "0x00000000",
            "size": "0x00"
          },
          {
            "address": "0x00000000",
            "size": "0x00"
          }
        ]
      },
```

```
      "boot_od_clk": {
        "description": "CM33 boot frequency",
        "applicable_conf": "48 MHz, 100 MHz, 180 MHz",
        "value": "180 MHz"
      }
    },
```

Table 3 displays the fields that users can configure.

Table 3 Boot policy

Field	Description
boot_cfg_id	Selects the boot configuration mode. SIMPLE_APP is a basic single-image boot, while SECURE_APP is a single image boot with secure boot enabled in the BootROM to validate the application Similarly, the DUAL_BANK_SIMPLE_APP, DUAL_BANK_SECURE_APP, and PROT_FW provide progressively stronger security and update capabilities
boot_bank_ctr_offset	The offset within each flash bank where the 32-bit bank-swap counter is stored. Used by the bootloader to determine which bank holds the most recently updated firmware
boot_hash_locking	When true, uses a faster hash comparison for boot authentication instead of full ECDSA signature verification. Recommended only where boot time is critical and the hash is stored securely
boot_auth_images	Controls which application images are authenticated at boot. ALL_MOVE authenticates all images and also moves them from Shared Secondary Slot (SSS) into execution memory after a successful DFU update
boot_auth_signature	The elliptic-curve signature algorithm used for image authentication. ES512 (ECDSA with P-521) provides the highest security level
boot_app_layout	Defines up to 5 application memory slots (address + size). Secure images use the 0x32xxxxxx address range; non-secure images use 0x22xxxxxx. Unused slots are set to 0x00000000
boot_od_clk	The CM33 core clock frequency during boot. Higher frequencies reduce boot time but increase power consumption

3.3 DFU policy

Device Firmware Update (DFU) is a mechanism that allows the firmware on a device to be updated over a wired communication interface (e.g., UART, SPI, I2C, etc.) without requiring physical access to debug/programming pins. It enables field upgrades, bug fixes, and feature additions after deployment. User has the flexibility to configure the SCB's dfu_pins_ctl for DFU pin control along with the DFU enable pin configuration dfu_en_pin.

To enable it, change "value": false to "value": true:

***Note**: The DFU is enabled by default in the device.*

3 Policy

DFU policy default configurations

```
"dfu": {
      "dfu_enable": {
        "description": "Indicates whether Device Firmware Update is enabled",
        "value": true
      },
      "dfu_if": {
        "description": "DFU interface",
        "applicable_conf": "UART, SPI, I2C",
        "value": "I2C"
      },
      "dfu_pins_ctl": {
        "description": "DFU pins control",
        "primary_port": {
          "description": "Primary DFU interface port number (0, 1, ..., 13)",
          "value": 2
        },
        "alternate_port": {
          "description": "Alternate DFU interface port number (0, 1, ..., 13), 0 for single-
port SCBs",
          "value": 0
        }
      },
      "dfu_en_pin": {
        "description": "DFU enable pin. Format: Px_y, where x is port number (0, 1, ..., 13)
and y is pin number (0, 1, ..., 7)",
        "value": "P2_2"
      },
      "dfu_config": {
        "description": "DFU interface configuration",
        "UART": {
          "scb": {
            "description": "SCB instance number (0-5)",
            "value": 0
          },
          "rx": {
            "description": "RX pin. num is the pin number (0-7); alternate_port is false for
the primary port or true for the alternate port",
            "num": 0,
            "alternate_port": false
          },
          "tx": {
            "description": "TX pin. num is the pin number (0-7); alternate_port is false for
the primary port or true for the alternate port",
            "num": 0,
            "alternate_port": false
          }
        },
        "SPI": {
          "scb": {
            "description": "SCB instance number (0-5)",
            "value": 0
```

```
            },
            "spi_clk_mode": {
              "description": "SPI SCLK mode (0-3)",
              "value": 0
            },
            "mosi": {
              "description": "MOSI pin. num is the pin number (0-7); alternate_port is false for
the primary port or true for the alternate port",
              "num": 0,
              "alternate_port": false
            },
            "miso": {
              "description": "MISO pin. num is the pin number (0-7); alternate_port is false for
the primary port or true for the alternate port",
              "num": 0,
              "alternate_port": false
            },
            "clk": {
              "description": "CLK pin. num is the pin number (0-7); alternate_port is false for
the primary port or true for the alternate port",
              "num": 0,
              "alternate_port": false
            },
            "nss": {
              "description": "NSS pin. num is the pin number (0-7); alternate_port is false for
the primary port or true for the alternate port",
              "num": 0,
              "alternate_port": false
            }
          },
          "I2C": {
            "scb": {
              "description": "SCB instance number (0-5)",
              "value": 1
            },
            "address": {
              "description": "I2C slave address, 7 bits (0-127)",
              "value": 53
            },
            "sda": {
              "description": "SDA pin. num is the pin number (0-7); alternate_port is false for
the primary port or true for the alternate port",
              "num": 2,
              "alternate_port": false
            },
            "scl": {
              "description": "SCL pin. num is the pin number (0-7); alternate_port is false for
the primary port or true for the alternate port",
              "num": 1,
              "alternate_port": false
            }
          }
```

```
    }
  },
```

3.4 Key-revocation policy

When the PSOC™ Control C3x6 device transitions to the Production lifecycle stage, only a single key revocation is permitted through the re-provisioning process. During ownership transfer, the OEM sends two OEM RoT keys to the device — `OEM_ROT_KEY_0` and `OEM_ROT_KEY_1` — so that if one key is compromised, the other remains available for OEM application validation.

By default, the BootROM uses `OEM_ROT_KEY_0` as the primary key for OEM application validation, in both the Development and Production LCS. If `OEM_ROT_KEY_0` is compromised while in the Production LCS, the OEM can re-provision the policy to revoke `OEM_ROT_KEY_0` and designate `OEM_ROT_KEY_1` as the primary key for BootROM-based OEM application validation.

Key revocation is limited to the Production LCS and cannot be performed in the Development LCS. It is also a one-time, irreversible process — once a key is revoked, the change is final, and no further revocation is possible.

To revoke the key, set its corresponding field value to true in the policy. The default policy configuration is shown in the following code block:

Key-revocation policy default configurations

```
"key_revocation": {
      "revoke_oem_rot_key_0": {
        "description": "Indicates whether to revoke OEM_ROT_KEY_0",
        "value": false
    },
```

3.5 Finalizing OEM assets

To lock in the policy parameters and prevent any further modifications, set the `prov_oem_complete` parameter to true in the `policy_oem_provisioning.json` file and provision the device.

```
"prov_oem_complete": {
  "description": "Indicates OEM assets are finalized in SFlash", "warning": "Once the
'prov_oem_complete' parameter value is set to true, it cannot be changed! No further updates
to SFlash are allowed",
"value": true
}
```

Warning: ***Once `prov_oem_complete` is set to true, the device is locked and cannot be provisioned again in Normal Provisioned LCS. This action is irreversible.***

3.6 Lifecycle stage (LCS) transitioning

The default LCS of the device out of factory is NORMAL_PROVISIONED. This is also called the development LCS. To transition the device to the Production LCS, set the `target_lcs` parameter to SECURE in the device_policy section of the `policy_oem_provisioning.json` file and provision the device.

```
"device_policy": {
  "target_lcs": {
    "description": "The LCS to which the transition should be done",
    "applicable_conf": "NORMAL_PROVISIONED, SECURE",
    "value": "SECURE"
  }
}
```

***Note**: The terms NORMAL_PROVISIONED LCS and DEVELOPMENT LCS are interchangeable; SECURE LCS is equivalent to PRODUCTION LCS.*

When transitioning the device to the Production LCS, it is recommended to set `prov_oem_complete` to true in the OEM provisioning policy so that all assets in SFlash are locked against further modification (Refer to Finalizing OEM assets). If the ability to re-provision the device during the Production LCS is still required, this field can instead be left set to false.

Warning**: **Once the device is moved to Production LCS, it is permanently locked into this stage and cannot be returned to Development LCS. The only further lifecycle transition available is to the Return Merchandise Authorization (RMA) LCS. Once the device is moved to RMA LCS, it cannot be returned to Production LCS nor Development LCS.

4 Edge Protect Tools

4.1 Overview

Edge Protect Tools is a Python-based package that provides a command-line interface for interacting with the MCU. It supports various functionalities, including provisioning, device integrity checks, RMA transition, and more. This tool is implemented in the Python programming language and uses several third-party Python modules to manage configuration files (`JSON`), object files (`ELF`), and perform cryptographic operations.

The tool can be accessed using any shell-based command-line interface. However, Infineon recommends using modus-shell, which is included in the ModusToolbox™ installation package, for seamless interaction with the Edge Protect Tools package.

When executing commands that require MCU interaction, such as provisioning, the tool prepares the necessary provisioning packets and communicates with the device using programming tools. It supports tools like OpenOCD via the SWD interface or the DFU Host Tool via serial interfaces, such as I^2C, SPI, or UART.

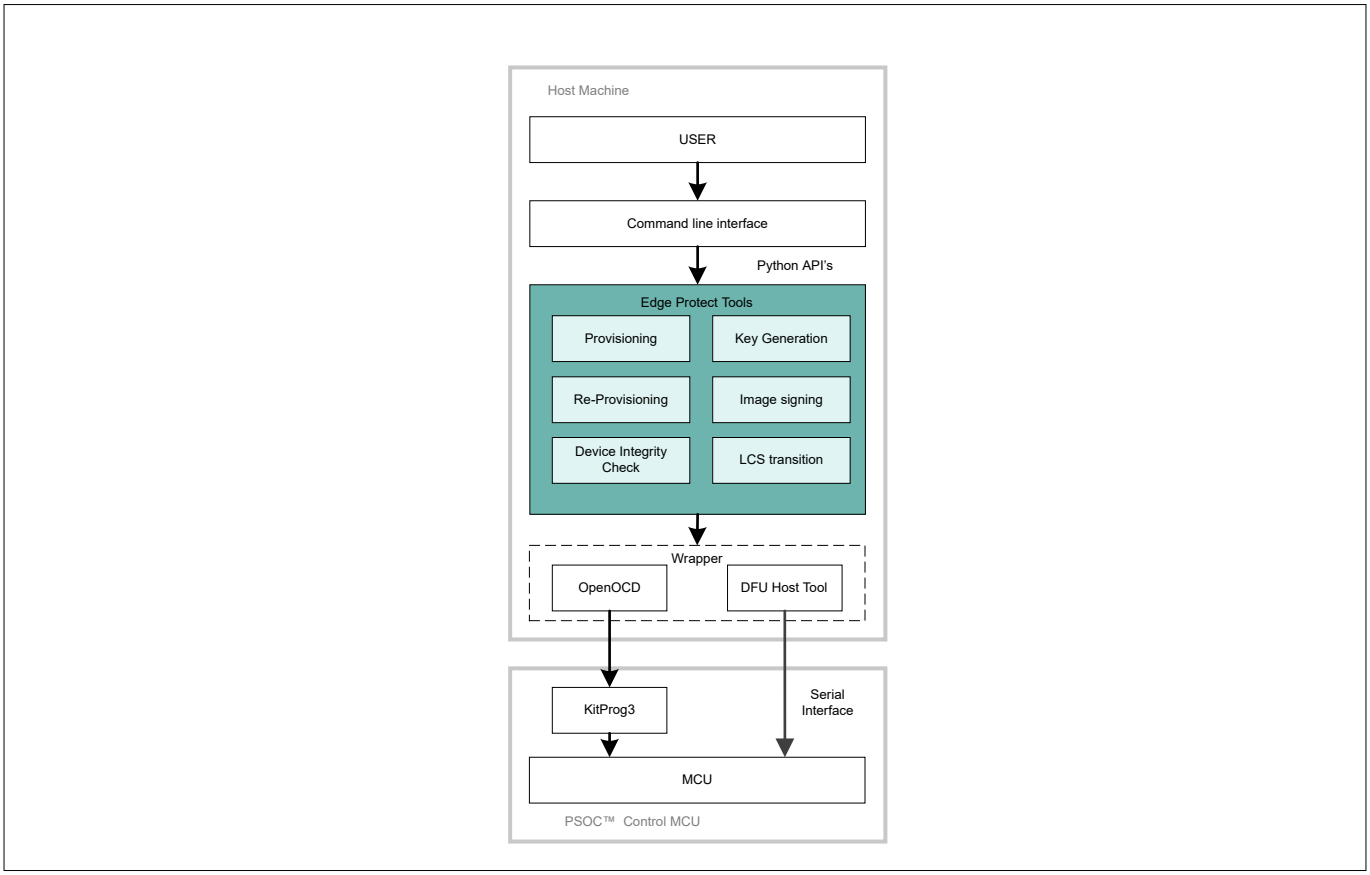


Figure 3 Edge Protect Tools overview

The operations supported by the tool include:

- **Provisioning**: Injecting keys and policies into the device
- **Re-provisioning**: Modifying specific configurations after provisioning
- **Key pair generation**: Creating key pairs for provisioning and image signing
- **Certificate generation**: Generating certificates for debugging and RMA transition
- **Signing images**: Signing images with the OEM key
- **Device Integrity Exam:** Verifies the authenticity of the device and ensures it is in a known state, thereby preventing any potential security breaches

For the complete list of commands supported by Edge Protect Tools package, see the Appendix A section.

4.2 Device communication

The Edge Protect Tools package supports communication with the device through programming tools such as OpenOCD via the SWD interface or the DFU Host Tool via serial interfaces like UART. The following table mentions the different interface support for various MCUs.

Table 4 Device communication interface support

Device	SWD	UART	I²C	SPI
PSOC™ Control C3 (P/M) 6x	Yes	Yes	Yes	Yes

Instead of directly invoking programming tool commands, the Edge Protect Tools package uses a wrapper mechanism to interface with these tools. This design is extendable, enabling potential integration with other programming tools.

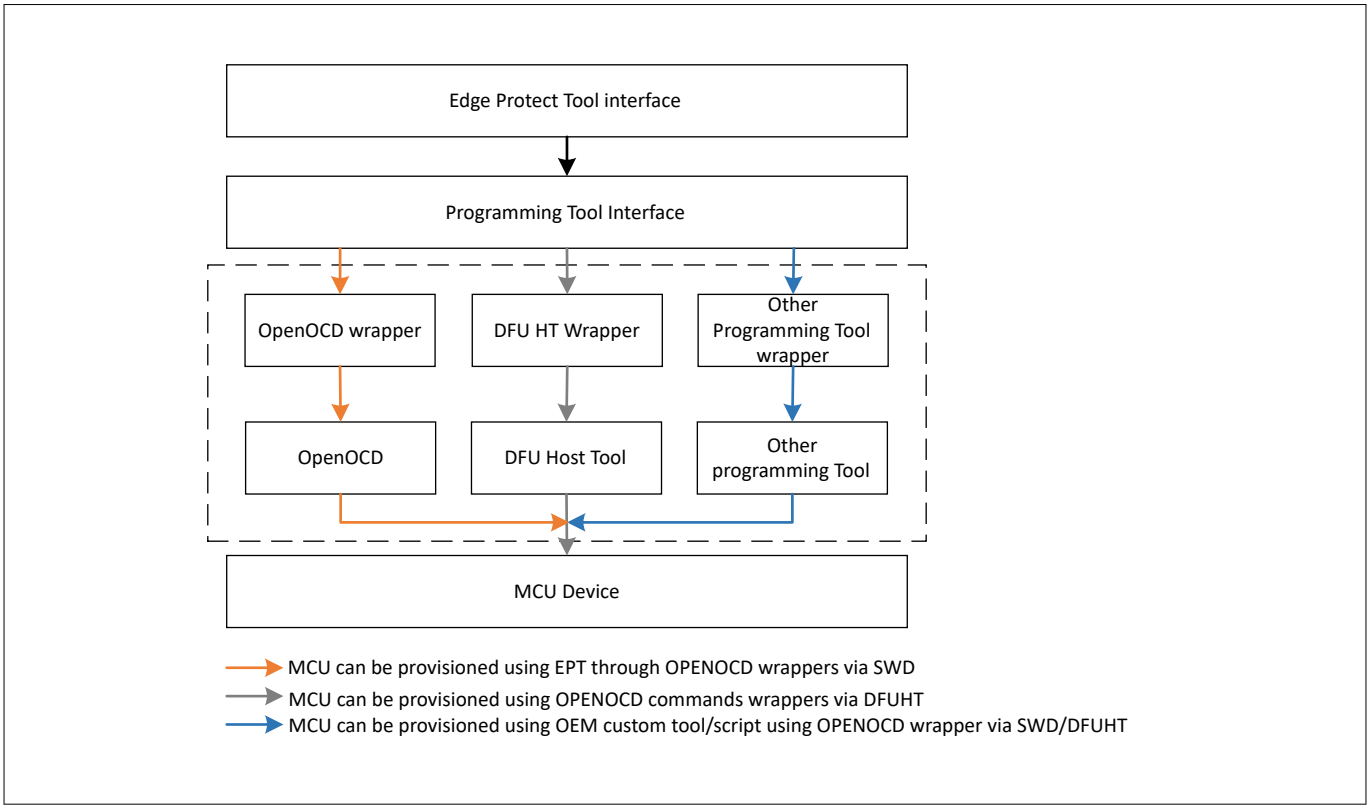


Figure 4 Edge Protect Tools device communication

4.3 Edge Protect Tools package for PSOC™ Control C3 (P/M) 6 MCU

Edge Protect Tools is included as part of the Edge Protect suite for the PSOC™ Control C3 (P/M) 6 MCU family.

Installation

Follow these steps to set up Edge Protect Security Suite:

1. Download and install the Infineon Developer Center Launcher
2. Log in using your Infineon credentials
3. Download and install the "ModusToolbox™ Edge Protect Security Suite" from the Developer Center Launcher. The EPT package is also included in the ModusToolbox™ setup

4 Edge Protect Tools

***Note**: The default installation directory of the Edge Protect Security Suite on Windows is `C:/Users/<USER>/Infineon/Tools`. After installing, add the Edge Protect Tools executable to your system PATH variable: `<ModusToolbox-Edge-Protect-Security-Suite-<version>>/tools/edgeprotecttools/bin`.*

***Note**: You need to install Python separately. Edge Protect Tools supports Python versions 3.8 to 3.12.*

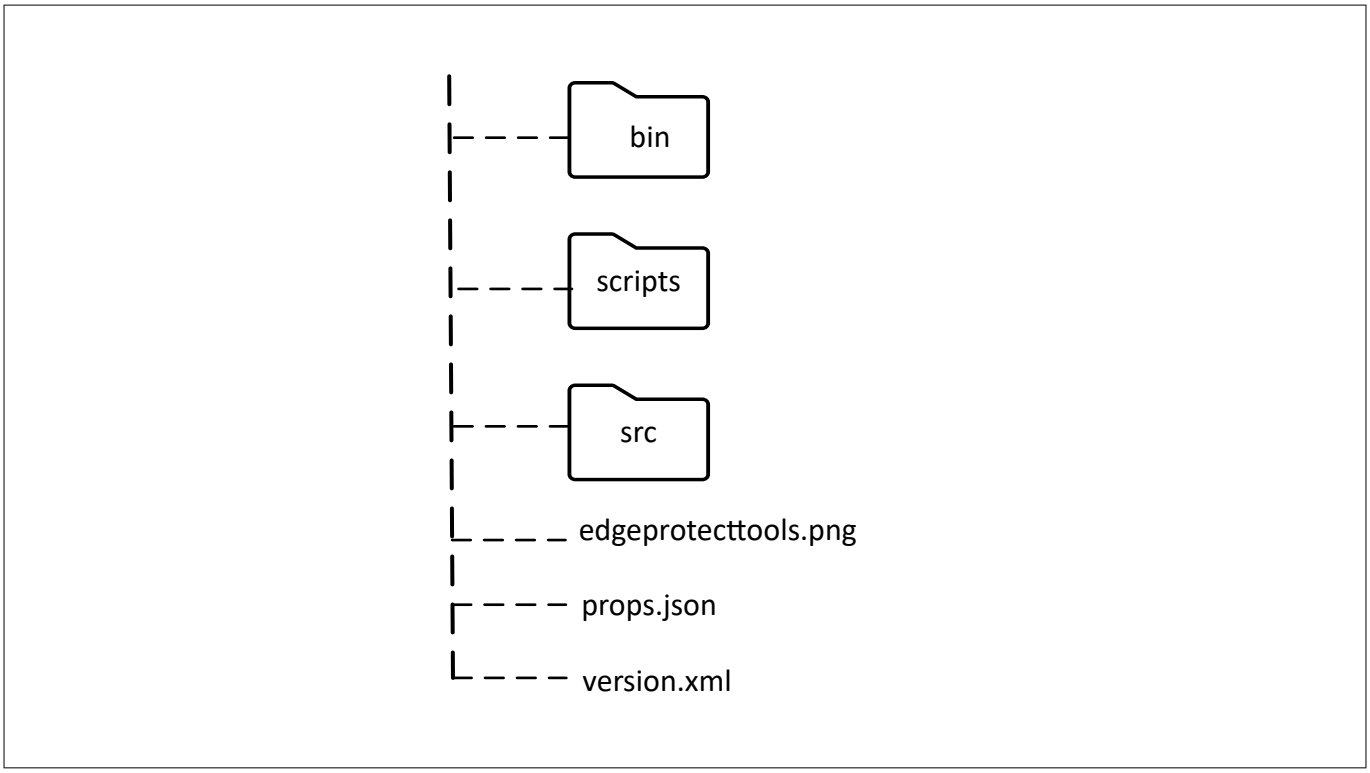


Figure 5 Edge Protect Tools in ModusToolbox™ Edge Protect Security Suite

The `./bin` directory contains a pre-compiled executable file for Windows, Linux, and macOS, which you can use directly. Add the executable path to your system's environment variable path:

```
<ModusToolbox-Edge-Protect-Security-Suite-install- path>/tools/edgeprotecttools/bin
```

Initialization

Before starting, Edge Protect Tools must be initialized for a specific target, such as the 'psoc_c3x6' for the PSOC™ Control C3 (P/M)(6)x devices.

To initialize, navigate to your working directory and execute the following command in a shell CLI:

```
edgeprotecttools -t psoc_c3x6 init
```

After initialization, the following folder structure as shown in Figure 6 will be created in your working directory.

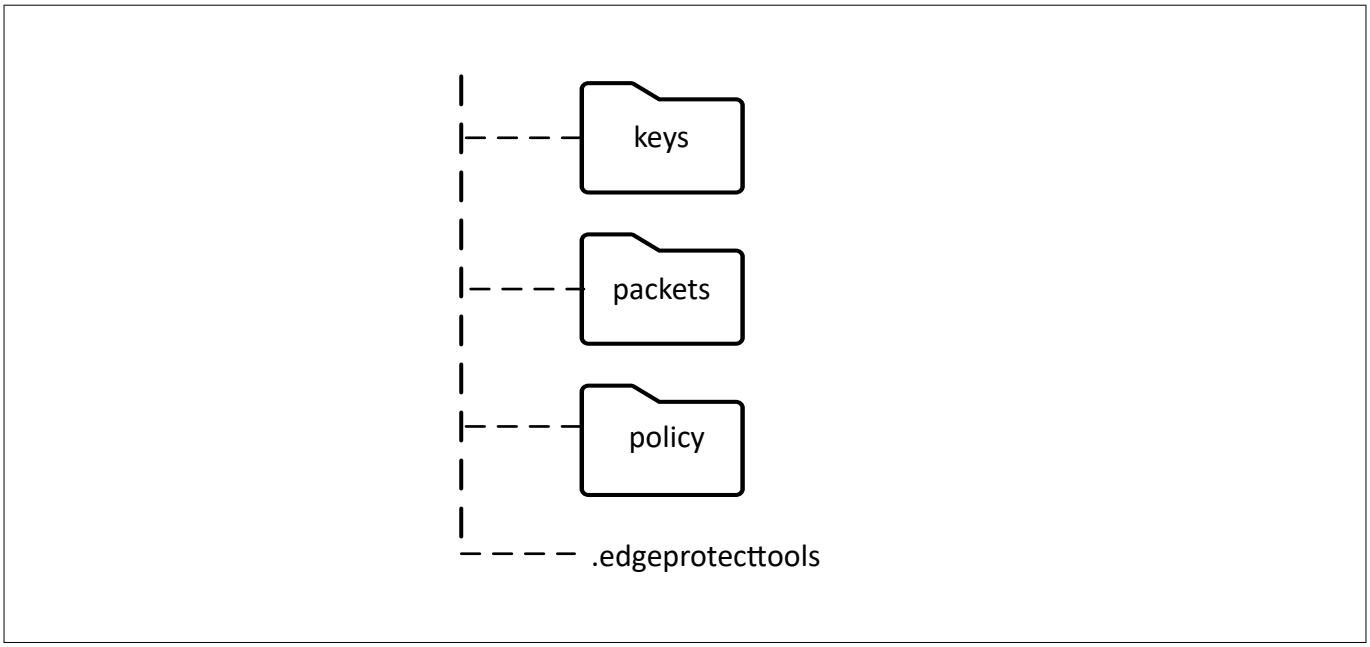


Figure 6 Edge Protect Tools project directory

Setting up the OpenOCD path:

To set up the OpenOCD path for use with the tools, execute the following command:

```
edgeprotecttools set-ocd --name openocd --path <openocd_path>
```

***Note**: Here, `<openocd_path>` is the path to the OpenOCD tool on your system. Typically, it will be located in the default installation directory: C:/Infineon/Tools/ModusToolboxProgtools-1.3/openocd.*

1. On Windows, use a Windows-style path (that is, not a path like `/cygdrive/c/`). Additionally, use forward slashes (/). For example, `C:/Users/XYZ/ModusToolbox/`
2. After installing Edge Protect Tools on Linux or macOS, follow any additional instructions provided by `pip` during the installation process

***Note**: If you are using modus-shell on a macOS system, the recommended usage for Edge Protect Tools is as follows: `arch-arm64 edgeprotecttools <command>`.*

4.3.1 Provisioning the device

To provision the device, execute the following command:

```
edgeprotecttools -t psoc_c3x6 provision-device -p policy/policy_oem_provisioning.json --key
keys/oem_rot_private_key.pem --ifx-oem-cert oem_cert.bin
```

If you are using an external MiniProg, first find its probe ID by running:

```
~/ModusToolbox/tools_3.3/fw-loader/bin/fw-loader.exe --device-list
```

This lists all connected devices along with their probe IDs. If you have multiple MiniProgs connected to your system, specify the correct probe ID in the provision-device command. For example:

```
edgeprotecttools -t psoc_c3x6 provision-device -p policy/policy_oem_provisioning.json --key
keys/oem_rot_private_key.pem --ifx-oem-cert oem_cert.bin --probe-id 061003B803260400
```

To finalize OEM assets and prevent further provisioning, or to transition the device to Production LCS, set the relevant parameters in the policy file before running the provision-device command.

See Lifecycle stage (LCS) transitioning for the `prov_oem_complete` and `target_lcs` policy parameters.

4.3.2 Packaged assets

The following assets are included in the Edge Protect Tools package and can be located in the `<working-dir>` where the Edge Protect Tools is initialized.

- **OEM policy**: The OEM policy file (`policy_oem_provisioning.json`) is used to configure device policy settings. This file is located in the `<working-dir>/policy` folder
- **RAM application**: Infineon-signed RAM application (DLM) binaries are available in the `<working-dir>/packets/apps` folder. RAM applications are owned by Infineon and cannot be customized by users
- **JSON template files**: JSON template files used by Edge Protect Tools are located in the `<working-dir>/packets` folder

5 Downloadable Module (DLM) package

A DLM package is a structured binary file that contains:

- Infineon signed RAM_APP
- Input parameters for device provisioning
- Signatures

5.1 RAM application (RAM_APP)

RAM applications are executable code loaded into the SRAM and executed in the PC0 (Protection Context 0) (AN243370 – Getting started with PSOC™ Control C3 (P/M) 6 security). Certain device features, such as provisioning and re-provisioning, are implemented using RAM applications. These applications are developed by Infineon, signed with Infineon keys, and verified and launched by the device's BootROM. It is important to note that RAM applications cannot be developed by OEMs.

RAM applications are packaged in the MCUboot image format. This format adds application metadata to verify image integrity and authenticity, along with custom metadata in the tag-length-value (TLV) format.

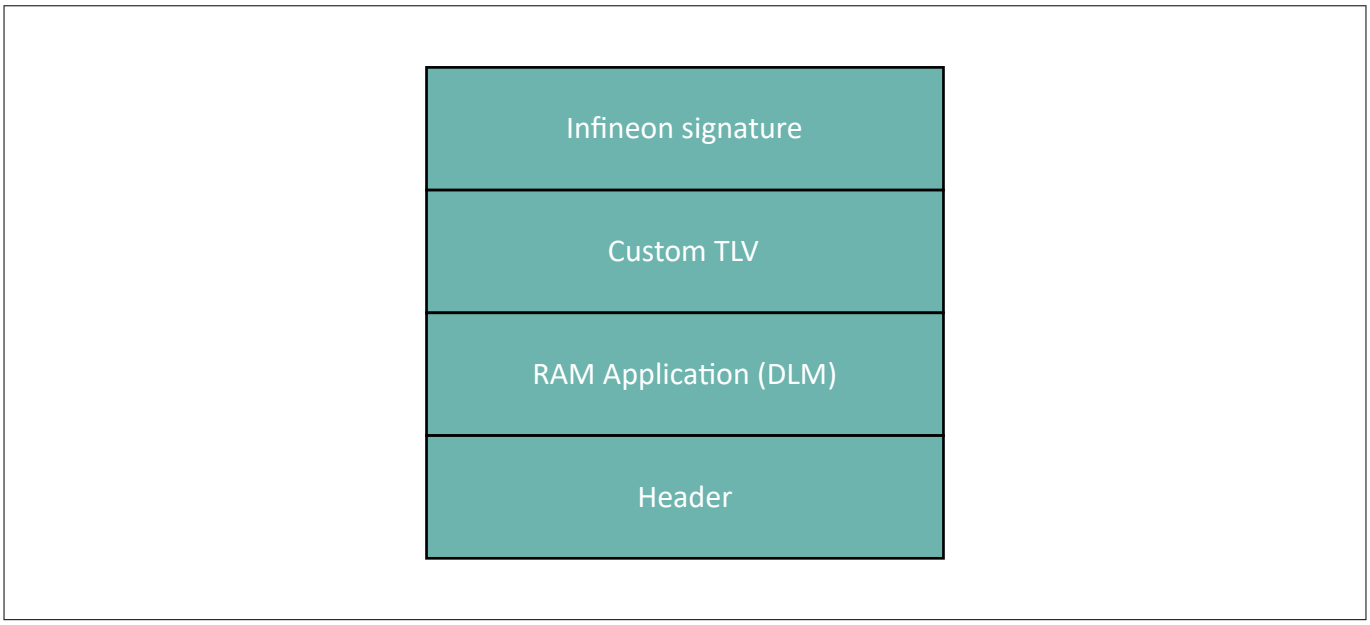


Figure 7 Signed RAM application

5.2 Downloadable Module (DLM) package

To provision a device, all the required input parameters required are set in the JSON files and this data is sent along with the RAM APP to the device to be provisioned. This whole package is called DLM package. During provisioning process, the edgeprotecttools transfer the package to the device SRAM, the device then launches the RAM_APP and the RAM_APP will provision the input parameters to the device SFlash. This package consists of the following components:

- Infineon signed RAM application
- DLM input parameters

The DLM packages are signed only for a few policies, such as reprovisioning, RMA, and DFU. For default device provisioning, the DLM signing is not required. The table below mentions which policies require signing.

Table 5 Policies requiring signatures

App name	Input parameters signed		DLM package signed	
	NORMAL_PROVISIONED LCS	SECURE LCS	NORMAL_PROVISIONED LCS	SECURE LCS
cyapp_prov_oem	NA	NA	NA	NA
cyapp_reprovisioning	Yes	Yes	Yes	Yes
cyapp_to_rma	NA	Yes	NA	Yes
cyapp_prot_fw_dfu	NA	NA	NA	Yes
cyapp_prot_fw_policy	NA	NA	NA	NA

***Note**: In the above table, the 'Yes' fields required OEM_ROT_KEYS for signing.*

To sign the DLM package, it is wrapped in the MCUboot format with a header and signature. The package is signed using the `OEM_ROT_PRIVATE_KEY`. The ROM boot always expects the DLM package to follow this format for processing; however, the package signature is verified only in the production lifecycle stage (SECURE LCS).

The OEM signature provides the OEM with control over which DLM is executed on the device in the production LCS, as well as its associated input parameters. To modify the input parameters within the package, the package must be re-signed using the `OEM_ROT_PRIVATE_KEY`.

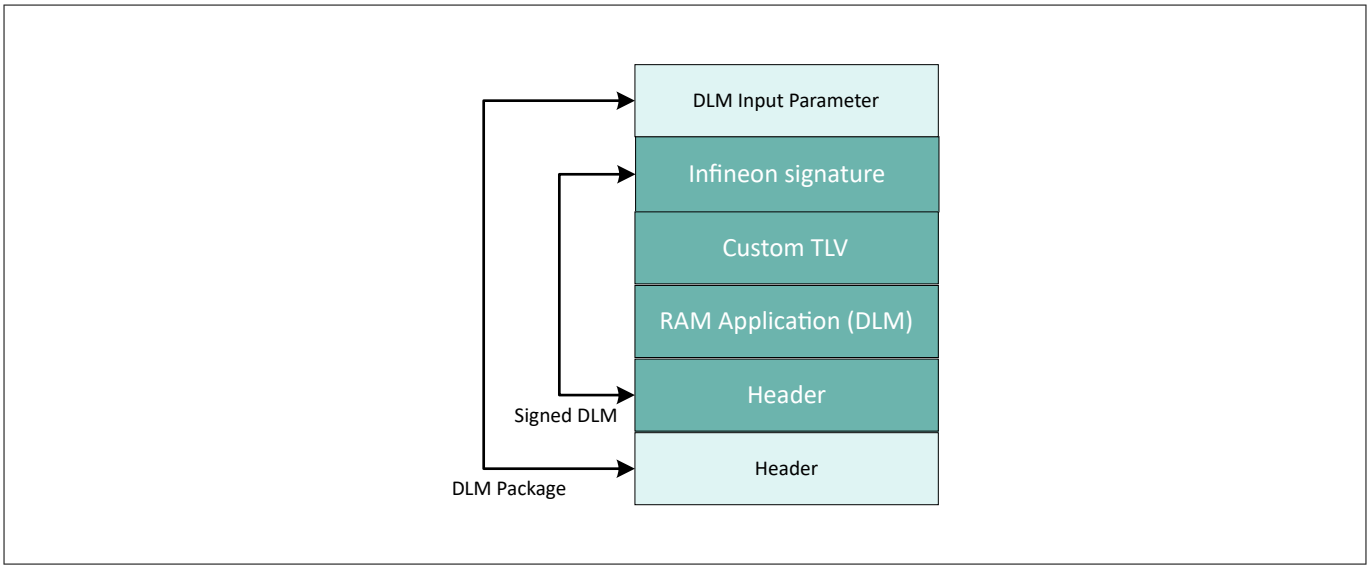


Figure 8 DLM package format

The placement of the DLM package in the device's SRAM memory is demonstrated in Figure 9. The DLM package is uploaded into the internal SRAM using either the debugger (SYS-AP) or an application firmware running on the device. Upon request, the BootROM validates the DLM package and post a successful validation, and it launches the RAM application in SRAM.

***Note**: In case of OEM provisioning, the DLM need not be signed and the DLM package will not contain the OEM signature.*

Once launched, the RAM application reads the input parameters from the device's SRAM and provisions them accordingly in the SFlash region.

The SRSS registers (BOOT_DLM_CTL) handle

- DLM package validation requests to BootROM

- RAM application launch control
- RAM application execution status reporting

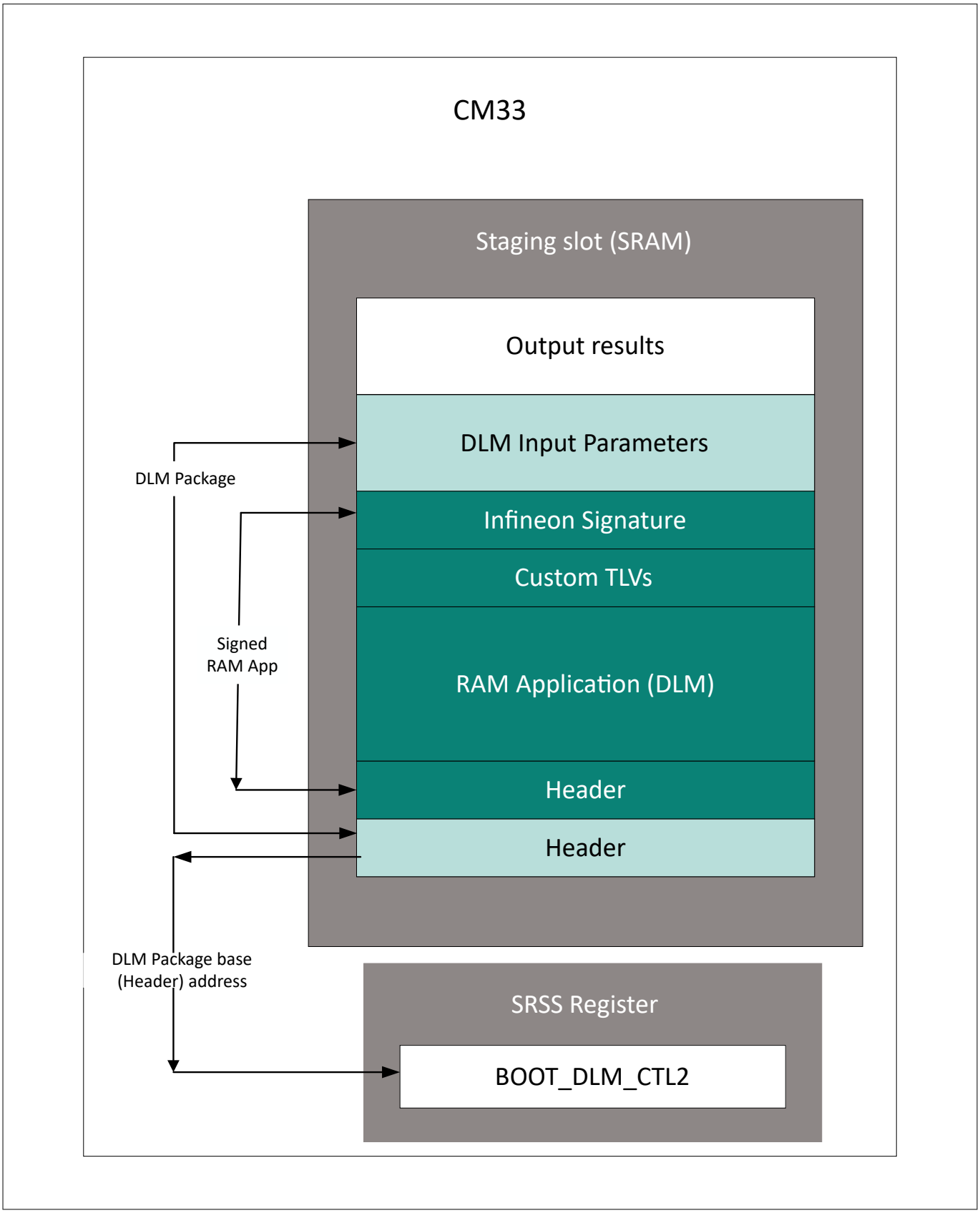


Figure 9 DLM package storage

Table 6 SRSS registers

SRSS register	Description
BOOT_DLM_CTL	Post requests for DLM package validation to the BootROM
BOOT_DLM_STATUS	Report the RAM application execution status in SRAM
BOOT_DLM_CTL_2	Store the DLM package base address in SRAM required by BootROM for validation
RES_SOFT_CTL	Trigger a device reset using the debugger (SYS-AP)

The input parameters, which are part of the DLM package placed in the device's SRAM, have their base address and maximum size stored by the ROM boot at a fixed SRAM location. This information is used by the RAM application to access the input parameters.

Similarly, the output results populated by the RAM application in the device's SRAM have their output result buffer base address and maximum size stored by the ROM boot at a fixed SRAM location. This information is used by the RAM application to write the output results.

See the Appendix A section for the list of RAM application status codes.

5.3 Creating a DLM package

Edge Protect Tools generates the DLM package from the pre-signed Infineon RAM application using the DLM parameters provided as input. The resulting DLM package includes the OEM signature, which is created using the OEM_ROT_PRIVATE_KEY. Additionally, the pre-signed Infineon RAM application binaries are included as part of the Edge Protect Tools package. Refer to the PSOC™ Control C3 MCU: OEM Provisioning code example for DLM package creation for a RAM application.

DLM package is created when *edgeprotecttools* provisioning command is run. It can be found in `<Workspace>/<CodeExampleName>/packets/apps/prov_oem/prov_oem_dlm.hex`

The DLM package can also be explicitly created with command below after running the `edgeprotecttools init` command.

```
# Create signed DLM package (the --key option may be skipped for BOOT_SIMPLE_APP)
$ edgeprotecttools -t psoc_c3x6 build-ramapp-package --app packets/apps/prov_oem/
cyapp_prov_oem_signed_ifx0.bin --inparams packets/apps/prov_oem/in_params.bin --key keys/
oem_rot_priv_key_0.pem --output packets/apps/prov_oem/prov_oem_dlm.hex
```

cyapp_prov_oem_signed_ifx0.bin - Infineon signed RAM_APP

in_params.bin - Input parameters to be provisioned to device

prov_oem_dlm.hex - Generated DLM HEX file

5.4 Download and launch of the DLM package

This section provides details on downloading and launching the RAM application using different methods available for the device.

Method #0 uses edgeprotecttools to download the launch the DLM using standard provisioning commands.

Method #1 uses BootROM DFU to download the DLM package to device.

5.4.1 Launch flow: BootROM DFU

Configuring the DFU Interface for Provisioning

EdgeProtectTools supports two interfaces for device provisioning:

- OpenOCD (default) — used for SWD-based communication
- Serial (DFU) — used for UART/I2C/SPI-based communication via the DFU protocol

To use the DFU interface instead of OpenOCD, complete the following two configuration steps before running any provisioning command:

Step 1: Set the serial interface as the active tool

```
edgeprotecttools set-ocd -n serial -p <PATH_TO_DFUH_TOOL>
```

For example:

```
edgeprotecttools set-ocd -n serial -p C:\Users\xxxx\ModusToolbox\packs\ModusToolbox-Industrial-MCU-Pack\tools\dfuh-tool
```

Step 2: Configure the serial port

Select the configuration command based on the protocol in use. Replace <id> with the MiniProg4 device ID (see note below).

UART configuration

```
$ edgeprotecttools serial-config --protocol uart --hwid MiniProg4-<id> --uart-baudrate 115200 --uart-databits 8 --uart-paritytype None --uart-stopbits 1
```

I2C configuration

```
$ edgeprotecttools serial-config --protocol i2c --hwid MiniProg4-<id> --i2c-address 12 --i2c-speed 400
```

SPI configuration

```
$ edgeprotecttools serial-config --protocol spi --hwid MiniProg4-<id> --spi-clockspeed 1 --spi-mode 00
```

***Note**: Finding the MiniProg4 device ID: Run the following command to list connected devices and retrieve the `MiniProg4 <id>`:*

`~/ModusToolbox/tools_3.3/fw-loader/bin/fw-loader.exe --device-list`

For a full list of serial-config options and parameters, refer to the EdgeProtectTools documentation.

Once both steps are complete, use the standard provisioning commands as usual — no further configuration is required. EdgeProtectTools will communicate with the device through the DFU protocol, and the BootROM will handle the communication on the device side.

Example:

Provisioning the OEM policy over DFU:

```
edgeprotecttools -t psoc_c3x6 provision-device -p .\policy\policy_oem_provisioning.json
```

To enable DFU communication on the device, configure the DFU policy parameters as described in DFU policy. For detailed information on the BootROM serial interface configuration, see the "BootROM serial interface" section in the PSOC™ Control C3 (P/M) 6 MCU architecture reference manual.

A Appendix

A.1 RAM application (DLM) status codes

The RAM application status code is written to the BOOT_DLM_STATUS register. This code is read by the Edge Protect Tools package to provide command execution logs.

Table 7 RAM application status codes

Code	Status name	Description
0x45000002UL	CYAPP_BAD_PARAM	One or more invalid parameters
0x45000003UL	CYAPP_LOCKED	Resource lock failure
0x45000004UL	CYAPP_STARTED	Operation started, but not necessarily completed yet
0x45000005UL	CYAPP_FINISHED	Operation finished
0x45000006UL	CYAPP_CANCELED	Operation canceled
0x45000007UL	CYAPP_TIMEOUT	Operation timed out
0x45000010UL	CYAPP_APP_RUNNING	The provisioning application is in progress
0x45000020UL	CYAPP_OTP_INIT_FAILED	Failed to initialize OTP
0x45000021UL	CYAPP_OTP_BOOTROW_WRITE_FAILED	Failed to update the LCS
0x45000022UL	CYAPP_OTP_BOOTROW_READ_FAILED	Failed to read the LCS
0x45000023UL	CYAPP_OTP_WRITE_FAILED	Failed to program object into the OTP
0x45000024UL	CYAPP_OTP_READ_FAILED	Failed to read object from the OTP
0x45000025UL	CYAPP_OTP_BOOTROW_CORRUPTED	BOOTROW is corrupted
0x45000026UL	CYAPP_NUM_ZEROS_INVALID	-
0x45000030UL	CYAPP_LCS_INVALID	The current device LCS is invalid for the execution of the RAM application
0x45000031UL	CYAPP_OEM_KEY_ALREADY_REVOKED	The OEM key 0 was revoked. This operation can only be performed once
0x45000032UL	CYAPP_ICV_KEY_ALREADY_REVOKED	The ICV key 0 was revoked. This operation can only be performed once
0x45000033UL	CYAPP_SIGNATURE_VERIF_FAILED	Failed to verify the input parameter's digital signature
0x45000034UL	CYAPP_KEY_0_ALREADY_PROGRAMMED	The OEM key 0 was programmed. This operation can only be performed once
0x45000035UL	CYAPP_KEY_1_ALREADY_PROGRAMMED	The OEM key 1 was programmed. This operation can only be performed once
0x45000036UL	CYAPP_OEM_ASSETS_ALREADY_PROGRAMMED	The OEM assets were programmed. This operation can only be performed once
0x45000037UL	CYAPP_HASH_CALCULATION_FAILED	Hash calculation failed
0x45000038UL	CYAPP_INVALID_BOOT_CFG_ID	Invalid BOOT_CFG_ID configuration

(table continues...)

Table 7 (continued) RAM application status codes

Code	Status name	Description
0x45000039UL	CYAPP_HUK_GEN_FAILED	Failed to generate the Wrap Master Key
0X4500003AUL	CYAPP_DFU_FAILED	DFU flow failed
0X4500003BUL	CYAPP_PROT_FW_POLICY_DISABLED	Protected firmware policy is not configured
0X4500003CUL	CYAPP_MPC_POLICY_INVALID	MPC_POLICY is invalid
0X4500003DUL	CYAPP_OTP_NOT_DEFAULT	OTP assets are not zero
0X4500003EUL	CYAPP_BAD_APP_LAYOUT	Application start address and/or size is invalid
0X4500003FUL	CYAPP_PROT_FW_KEY_ALREADY_REVOKED	The PROT_FW key 0 was revoked. This operation can only be performed once
0x4500003EUL	CYAPP_HASH_CALCULATION_FAILED	Failed to calculate hash
0x4500003FUL	CYAPP_PROVISIONING_LIMITS_FAILED	Provisioning limits exceeded
0x45000100UL	CYAPP_PARAM_NV_CNT_INVALID	The input parameter NV counter is not valid
0x450000101UL	CYAPP_PARAM_OEM_KEY_0_HASH_INVALID	The input parameter OEM key 0 hash is not valid
0x450000102UL	CYAPP_PARAM_OEM_KEY_1_HASH_INVALID	The input parameter OEM key 1 hash is not valid
0x450000103UL	CYAPP_PARAM_PUBKEY_INVALID	The OEM public key used for digital signature verification does not match the OEM public key hash provisioned in the OTP
0x450000104UL	CYAPP_PARAM_CONTROL_WORD_INVALID	The input parameter control word is not valid
0x450000105UL	CYAPP_PARAM_TARGET_LCS_INVALID	The input parameter target LCS is not valid. Valid values are NORMAL, NORMAL_NO_SECURE, and SECURE
0x450000106UL	CYAPP_RMA_CERT_VERIF_FAILED	Failed to verify the certificate for RMA LCS transition
0x450000107UL	CYAPP_PARAM_ACCESS_RESTRICT_INVALID	The input parameter ACCESS_RESTRICT has an invalid configuration
0x450000108UL	CYAPP_RMA_ATTEMPT_COUNTER_INVALID	RMA_ATTEMPT_COUNTER is equal to zero
0x450000109UL	CYAPP_IAK_ALREADY_PROGRAMMED	IAK already programmed
0x45000010AUL	CYAPP_BOOTROW_CHECK_FAILED	BootROW check failed
0x45000010BUL	CYAPP_EFUSE_CHECK_FAILED	eFuse data check failed
0x45000010CUL	CYAPP_CBOR_INVALID	CBOR packet is invalid
0X4500010DUL	CYAPP_PROT_FW_POLICY_INVALID	PROT_FW_POLICY is invalid after decryption

(table continues...)

Table 7 (continued) RAM application status codes

Code	Status name	Description
0xF2A00001UL	CYAPP_SUCCESS	The RAM application is completed successfully
0xF2A00010UL	CYAPP_APP_RUNNING	The RAM application is in progress

A.2 Edge Protect Tools supported commands

Table 8 List of supported commands for PSC3 target device

Command	Description
bin2hex	Converts a binary image to a hex image
build-ramapp-package	Builds the DLM package with the RAM application and input parameters
cbor2json	Converts CBOR to JSON format
convert-key	Converts key to other formats
cose-sign	Creates a COSE_Sign message
cose-sign1	Creates a COSE_Sign1 message
cose-verify	Verifies the COSE_Sign message
create-key	Creates private and public key pairs
create-provisioning-packet	Creates a binary packet for device provisioning
debug-token	Creates debug token binary-based on template
device-info	Reads device information - Silicon ID, Family ID, Silicon revision
device-list	Lists the supported devices
encrypt-aes	Encrypts binary files using AES encryption
hash	Calculates hash of a file
hex-segment	Extracts a segment from the hex. file
hex2bin	Converts hex image to binary image
init	Initializes a new project
integrity-exam	Executes the integrity exam app
merge-bin	Merges different binary files into one
merge-hex	Merges different hex files into one
provision-device	Executes device provisioning
read-die-id	Reads die ID from the device
reprovision-device	Executes device re-provisioning
rma-token	Creates 'transit to RMA' or 'open RMA' token
serial-config	Configures serial interfaces

(table continues...)

Table 8 (continued) List of supported commands for PSC3 target device

Command	Description
serial-ports	Lists connected compatible hardware
set-ocd	Sets on-chip debugger
transit-to-rma	Transitions device to RMA lifecycle stage
verify-image	Verifies image with a key
verify-packet	Verifies provisioning packet with a key
version	Shows firmware version

To view list of commands supported by this specific device, execute the following in the shell CLI:

```
edgeprotecttools -t psoc_c3x6 --help
```

To view a list of options for a specific command, execute the following in the shell CLI:

```
edgeprotecttools -t psoc_c3x6 <COMMAND> --help
```

A.3 CBOR and COSE format

The Concise Binary Object Representation (CBOR) is a data format used in PSOC™ Control devices for concise data representation. The CBOR data format is defined as an Internet standard by RFC 8949, while the CBOR Object Signing and Encryption (COSE) format is defined in RFC 8152. COSE is used to ensure authentication, integrity, and confidentiality of exchanged data. Based on the JSON data model, the CBOR/COSE format is used to transport binary data such as certificates, firmware images, policy files, etc. by the Edge Protect Tools. Additionally, the CBOR web token (CWT), an open industry standard defined in RFC 8392 is used for wrapping a CBOR packet. For example, policy files are represented in JSON format. During the creation of a provisioning packet, the Edge Protect Tools converts the policy to CBOR, then wrap the payload to CWT for use as RAM application input parameters. When a COSE object is used, the optional parameter 'cose-type' can be used to identify the embedded object. The value to use with the parameter for each of the structures is defined in Table 9.

Table 9 CBOR tag and cose-type

CBOR tag	cose-type	Data item	Semantics
98	cose-sign	COSE-Sign	COSE Signed data object
18	cose-sign1	COSE_Sign1	COSE Single signer data object
96	cose-encrypt	COSE_Encrypt	COSE Encrypted data object
16	cose-encrypt0	COSE_Encrypt0	COSE Single recipient encrypted data object
97	cose-mac	COSE_Mac	COSE Mac data object
17	cose-mac0	COSE_Mac0	COSE Mac without recipient object

Edge Protect Tools can be used for conversion from CBOR to JSON format to create COSE_Sign and COSE_Sign1 messages, and to verify a COSE_Sign message. See Appendix A.2.

References

Contact Infineon support to obtain these documents.

Application notes:

[1] Infineon Technologies AG: AN243049 – Getting started with PSOC™ Control C3 (P/M) 6 on ModusToolbox™ software

[2] Infineon Technologies AG: AN243370 – Getting started with PSOC™ Control C3 (P/M) 6 C3 security

Datasheets:

[3] Infineon Technologies AG: 002-42056: PSOC™ Control C3 - PSC3P6xx, PSC3M6xx

Reference manuals:

[4] Infineon Technologies AG: 002-42262: PSOC™ Control C3 architecture reference manual (PSC3P6xx, PSC3M6xx)

[5] Infineon Technologies AG: 002-42706: PSOC™ Control C3 registers reference manual (PSC3P6xx, PSC3M6xx)

Glossary

ASF
Alternate serial interface

CA
Certificate authority

CBOR
Concise Binary Object Representation. See the CBOR webpage.

CLI
Command-line interface

COSE
CBOR Object Signing and Encryption

CoT
Chain of Trust

CSR
Certificate signing request

CWT
CBOR web token

DFU
Device Firmware Update

DFU HT
Device Firmware Update Host Tool

DLM
Downloadable module. Alternatively referred to in this document as RAM application

DLM package
Package consisting of the RAM application (DLM), input parameters, and the OEM signature

EPC 2
Edge Protect Category 2. See Infineon Edge Protect.

FW
Firmware

GPIO
General-purpose input/output

I2C
Inter-integrated circuit

LCS
Lifecycle stage

Glossary

MCU
Microcontroller unit

MFG
Manufacturing

MPC
Memory protection controller

NVM
Non-volatile memory

OEM
Original Equipment Manufacturer. Infineon's customers.

OEM_ROT_DEV_PRIVATE_KEY
Private part of the OEM_ROT development key pair

OEM_ROT_DEV_PUBLIC_KEY
Public part of the OEM_ROT development key pair

OEM_ROT_KEY
OEM Root of Trust key pair

OEM_ROT_PRIVATE_KEY
Private part of the OEM_ROT key pair.

OEM_ROT_PUBLIC_KEY
Public part of the OEM_ROT key pair.

OTP
One-time programmable

PC
Personal computer

PEM
Privacy Enhanced Mail

RAM application
RAM applications are developed, signed by Infineon, and loaded into SRAM for the purpose of provisioning and LCS transition.

RMA
Return merchandise authorization

RoT
Root of Trust

SPI
Serial Peripheral Interface

Glossary

SRSS
System resources subsystem

SWD
Serial Wire Debug

TLV
Tag-length-value

UART
Universal Asynchronous Receiver Transmitter

USB
Universal Serial Bus

Revision history

Document revision	Date	Description of changes
**	2026-07-30	Initial release

Trademarks

PSOC™, formerly known as PSoC™, is a trademark of Infineon Technologies. Any references to PSoC™ in this document or others shall be deemed to refer to PSOC™.

Trademarks

All referenced product or service names and trademarks are the property of their respective owners.

Edition 2026-07-30

Published by

Infineon Technologies AG

81726 Munich, Germany

© 2026 Infineon Technologies AG

All Rights Reserved.

Do you have a question about any aspect of this document?

Email: erratum@infineon.com

Document reference

IFX-vjt1784007811218

Important notice

Products which may also include samples and may be comprised of hardware or software or both ("Product(s)") are sold or provided and delivered by Infineon Technologies AG and its affiliates ("Infineon") subject to the terms and conditions of the frame supply contract or other written agreement(s) executed by a customer and Infineon or, in the absence of the foregoing, the applicable Sales Conditions of Infineon. General terms and conditions of a customer or deviations from applicable Sales Conditions of Infineon shall only be binding for Infineon if and to the extent Infineon has given its express written consent.

For the avoidance of doubt, Infineon disclaims all warranties of non-infringement of third-party rights and implied warranties such as warranties of fitness for a specific use/purpose or merchantability. Infineon shall not be responsible for any information with respect to samples, the application or customer's specific use of any Product or for any examples or typical values given in this document.

If this document is marked as 'Preliminary', 'Target', 'Draft' or 'Proposal', Infineon reserves the right to change all information given in this document at any time without notice. Subject to the development and release of a Product for series supply by Infineon, the technical specifications of the Product are set forth in the relevant datasheet provided by Infineon without such marking.

The data contained in this document is exclusively intended for technically qualified and skilled customer representatives. It is the responsibility of the customer to evaluate the suitability of the Product for the intended application and the customer's specific use and to verify all relevant technical data contained in this document in the intended application and the customer's specific use. The customer is responsible for properly designing, programming, and testing the functionality and safety of the intended application, as well as complying with any legal requirements related to its use.

Unless otherwise explicitly approved by Infineon, Products may not be used in any application where a failure of the Products or any consequences of the use thereof can reasonably be expected to result in personal injury.

However, the foregoing shall not prevent the customer from using any Product in such fields of use that Infineon has explicitly designed and sold it for, provided that the overall responsibility for the application lies with the customer. Infineon expressly reserves the right to use its content for commercial text and data mining (TDM) according to applicable laws, e.g. Section 44b of the German Copyright Act (UrhG). If the Product includes security features:

Because no computing device can be absolutely secure, and despite security measures implemented in the Product, Infineon does not guarantee that the Product will be free from intrusion, data theft or loss, or other breaches ("Security Breaches"), and Infineon shall have no liability arising out of any Security Breaches.

If this document includes or references software:

The software is owned by Infineon under the intellectual property laws and treaties of the United States, Germany, and other countries worldwide. All rights reserved. Therefore, you may use the software only as provided in the software license agreement accompanying the software.

If no software license agreement applies, Infineon hereby grants you a personal, non-exclusive, non-transferable license (without the right to sublicense) under its intellectual property rights in the software (a) for software provided in source code form, to modify and reproduce the software solely for use with Infineon hardware products, only internally within your organization, and (b) to distribute the software in binary code form externally to end users, solely for use on Infineon hardware products. Any other use, reproduction, modification, translation, or compilation of the software is prohibited.

For further information on the Product, technology, delivery terms and conditions, and prices, please contact your nearest Infineon office or visit www.infineon.com.